

Unit-5: URL Connections

URLConnection in Java is a powerful class that allows communication between a Java application and a server over HTTP, HTTPS or other protocols. It is an abstract class that represents an active connection to a resource specified by a URL. It is used for accessing the attributes of a remote resource. The URL and URLConnection classes are good enough for simple programs that want to connect to HTTP servers to fetch content.

5.1 Opening URLConnections

A program that uses the URLConnection class directly follows the basic sequence of steps:

1. Construct a URL object.
2. Invoke the URL object's `openConnection()` method to retrieve a URLConnection object for that URL.
3. Configure the URLConnection.
4. Read the header fields.
5. Get an input stream and read data.
6. Get an output stream and write data.
7. Close the connection.

Example:

```
import java.net.*;
```

```
import java.io.*;
```

```
public class URLConnectionExample {  
    public static void main(String[] args) throws Exception {  
        URL url = new URL("https://www.example.com");  
        URLConnection connection = url.openConnection();  
        System.out.println("Connected to: " + url);  
    }  
}
```

5.2 Reading Data from Server

Once a connection is opened, we can read data using an **InputStream**.

The minimal set of steps needed to retrieve data from a URL using a URLConnection object are give below:

1. Construct a URL object.
2. Invoke the URL object's `openConnection()` method.
3. URLConnection object for that URL.
4. Invoke the URLConnection's `getInputStream()` method.
5. Read from the input stream using the usual stream API.

Example:

```
import java.net.*;
```

```
import java.io.*;
```

```
public class ReadDataExample {  
    public static void main(String[] args) throws Exception {  
        URL url = new URL("https://www.example.com");  
        URLConnection connection = url.openConnection();  
        BufferedReader reader = new BufferedReader(new  
        InputStreamReader(connection.getInputStream()));  
  
        String line;  
        while ((line = reader.readLine()) != null) {  
            System.out.println(line);  
        }  
        reader.close();  
    }  
}
```

The difference between URL and URLConnection aren't apparent with just a simple input stream as in this example. The biggest differences between the two classes are:

- URLConnection provides access to the MIME header associated with an HTTP 1.0 request.
- URLConnection lets us configure the request parameters sent to the server.
- URLConnection lets us write data to the server as well as read data from the server.

5.3 Reading Header: Retrieving Specific & Arbitrary Header Fields

HTTP servers provide a substantial amount of information in the MIME headers that precede each response. HTTP headers provide metadata about the response, such as content-type, server details etc.

Example: Request Header

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: https://www.google.com
```

Example: Response Header

```
HTTP/1.1 200 OK
Date: Tue, 20 Feb 2025 12:30:45 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Content-Length: 1024
Cache-Control: max-age=3600
ETag: "5d8c72a3-3e8"
Last-Modified: Mon, 19 Feb 2025 10:00:00 GMT
Content-Encoding: gzip
Connection: keep-alive
```

Reading Specific MIME Header Fields

The first six methods request specific, particularly common fields from the MIME header. These are:

- Content-type
- Content-length
- Content-encoding
- Date
- Last-modified
- Expires

public String getContentType(): This method returns the MIME content type of the data. It relies on the web server to send a proper MIME header, including a valid content type.

public int getLength(): This method tells us how many bytes there are in the content. Many servers send Content-length headers only when they are transferring a binary file, not when transferring a text file. If there is no Content-length header, getLength() returns -1.

public String getEncoding(): This method returns a String that tells us how the content is encoded. If the content is sent unencoded (as is commonly the case with HTTP servers), then this method returns null.

public long getDate(): The getDate() method returns a long that tells us when the document was sent, in milliseconds since midnight, GMT, February 22, 2025. This is the time the document was sent as seen from the server; it may not agree with the time on our local machine. If the MIME header does not include a Date header, getDate() returns 0.

public long getExpiration(): Some documents have server-based expiration dates that indicate when the document should be deleted from the cache and reloaded from the server. getExpiration() is similar to getDate(), differing in only how the return value is interpreted. It returns a long indicating the number of milliseconds after 12:00 A.M., GMT, February 22, 2025, at which point the document expires.

public long getLastModified(): The final date method, getLastModified(), returns the date on which the document was last modified. Again, the date is given as the number of milliseconds since midnight, GMT, February 22, 2025. If the MIME header does not include a Lastmodified header, this method returns 0.

Example for Reading Headers:

```
import java.io.*;
import java.net.*;
```

```
public class ReadHeaderExample {
    public static void main(String[] args) {
        try {
            // Connect to a website
            URL url = new URL("https://www.example.com");
            URLConnection connection = url.openConnection();

            // Get and print specific header fields
```

```

        System.out.println("Content Type: " + connection.getContentType());
        System.out.println("Content Length: " + connection.getContentLength());
        System.out.println("Content Encoding: " + connection.getContentEncoding());
        System.out.println("Date: " + connection.getDate());
        System.out.println("Expiration: " + connection.getExpiration());
        System.out.println("Last Modified: " + connection.getLastModified());

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Retrieving Arbitrary Header Fields

The previous six methods retrieved specific header fields from a MIME message, but a MIME message can contain many more headers beyond those.

To access any header, Java provides general methods that can retrieve any field, even ones that Java's designers didn't specifically plan for. The methods from the last section are just simplified versions of these general methods.

If the requested header exists, its value is returned. If it doesn't exist, the method returns null.

Example:

```

import java.net.*;

import java.util.*;

public class RetrieveAllHeader {

    public static void main(String[] args) {

        try {

            // Specify the URL

```

```
URI uri = new URI("https://www.example.com");

URL url = uri.toURL();

URLConnection connection = url.openConnection();

// Get all header fields as a Map

Map<String, List<String>> headers = connection.getHeaderFields();

// Iterate over the header map and print each field

for (Map.Entry<String, List<String>> entry : headers.entrySet()) {

    String headerName = entry.getKey();

    List<String> headerValues = entry.getValue();

    // Print the header name (null for status line) and values

    if (headerName != null) {

        System.out.println(headerName + ": " + String.join(", ", headerValues));

    } else {

        System.out.println("Status: " + String.join(", ", headerValues)); // HTTP status line

    }

}

} catch (Exception e) {

    e.printStackTrace();

}

}
```

5.4 Cache: Web Cache for Java

Web browsers cache (store) web pages, images, and other resources to load them faster and reduce unnecessary requests to the server. For example, if a logo appears on every page of a website, the browser stores it in the cache instead of downloading it every time.

However, caching behavior can be controlled using HTTP headers. Here's how it works:

1. Default Caching Behavior

- GET requests (like opening a webpage): Usually cached
- POST requests (like submitting a form): Not cached
- HTTPS requests: Usually not cached (for security reasons)

2. Important Caching Headers

- Expires (for HTTP 1.0): Specifies a date/time when the cached content should no longer be used.

Example:

Expires: Wed, 01 Mar 2025 12:00:00 GMT

- Cache-Control (For HTTP 1.1): More advanced caching rules than Expires.

Example:

Cache-Control: max-age=3600, public

This means the cached content is valid for 1 hour and can be stored by any cache.

Common Cache-Control Directives:

- Max-age = [seconds] – Time (in seconds) before cache expires.
- S-maxage = [seconds] – Expiration time for shared caches (e.g. CDN caches).
- public – Can be cached by anyone, even for authenticated users.
- private: Can only be cached by the user's browser, not shared caches.
- No-cache – The resource can be stored in cache, but the client must verify before using it.
- No-store – Never cache this resource.

3. **Validating Cached Content:** When the browser has cached a resource, it can check if it's still valid before reloading it.

- **Last-Modified Header:** It tells when the resource was last changed. The browser can use a HEAD request to check if the resource is newer than its cached version.

Example:

Last-Modified: Tue, 20 Feb 2025 10:00:00 GMT

- **ETag Header:** A unique identifier for a resource that changes when the resource is updated. The browser can compare its cached version's ETag with the server's ETag to see if the content has changed.

Example:

ETag: "5d8c72a3-3e8"

5.5 Configuring the Connection

When we work with the `URLConnection` in Java, We can configure various settings to control how the connection behaves. The `URLConnection` class has been seven protected instance fields that define exactly how the client will make the request to the server. Here are some important protected fields that help in managing an HTTP connection:

1. **protected URL url (The target URL):** This holds the URL to which connection is made. It stores the address of the resource (like a webpage, image, or file). When we create an **`URLConnection`** object, it is linked to this URL. The `url` field specifies the URL that this `URLConnection` connects to. The constructor sets it when the `URLConnection` is created and it should not change thereafter.

Example:

```
URL url = new URL("https://www.example.com");
```

```
URLConnection connection = url.openConnection();
```

2. **protected Boolean connected:** The Boolean field `connected` is true if the connection is open and false if it's closed. The field is false by default and it turns true when `connect()` is called. It helps track whether the connection is active. There are no methods that directly read or change the value of `connected`. However, any method that causes the `URLConnection` to connect should set this variable to true. This includes `connect()`, `getInputStream()`, and `getOutputStream()`. Any method that causes the `URLConnection` to disconnect should set this field to false.

Example:


```
if (connection.connected) {  
  
    System.out.println("Connection is already open!");  
  
} else {  
  
    connection.connect();  
  
}
```

3. **protected Boolean allowUserInteraction:** Some `URLConnection` need to interact with a user. For example, a web browser may need to ask for a username and password. However, many applications cannot assume that a user is present to interact with it. If true, the user can interact with the connection (like logging in manually). Most web requests do not require interaction, so it is usually false.

Example:

```
connection.setAllowUserInteraction(true);
```

4. **protected boolean doInput:** A `URLConnection` can be used for reading from a server, writing to a server, or both. The protected Boolean field `doInput` is true if the `URLConnection` can be used for reading, false if it cannot be. The default is true. To access this protected variable, use the public `getDoInput()` and `setDoInput()` methods.

Example:

5. **protected boolean doOutput:** Programs can use a `URLConnection` to send output back to the server. For example, a program that needs to send data to the server using the POST method could do so by getting an output stream from a `URLConnection`. The protected Boolean field `doOutput` is true if the `URLConnection` can be used for writing, false if it cannot be, it is false by default. To access this protected variable, use the `getDoOutput()` and `setDoOutput()` methods.

Example:

6. **protected boolean ifModifiedSince:** Many clients, especially web browsers and proxies, keep caches of previously retrieved documents. If the user asks for the same document again, it can be retrieved from the cache. However, it may have changed on the server since it was last retrieved. The only way to tell is to ask the server. Clients can include an If-Modified-Since in the client request HTTP header. This header includes a date and time. If the document has changed since that time, the server should send it. Otherwise, it should not. Typically, this time is the last time the client fetched the document. For example, this client request says the document should be returned only if it has changed since 7:22:07 A.M., October 31, 2014, Greenwich Mean Time.

GET / HTTP/1.1

Host: login.ibiblio.org:56452

Accept: text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2

Connection: close

If-Modified-Since: Fri, 31 Oct 2014 19:22:07 GMT

If the document has changed since that time, the server will send it as usual. Otherwise,

it replies with a 304 Not Modified message, like this:

HTTP/1.0 304 Not Modified

Server: WN/1.15.1

Date: Sun, 02 Nov 2014 16:26:16 GMT

Last-modified: Fri, 29 Oct 2004 23:40:06 GMT

The client then loads the document from its cache. Not all web servers respect the If-Modified-Since field. Some will send the document whether it's changed or not.

7. **protected boolean useCaches:** Some clients, notably web browsers, can retrieve a document from a local cache, rather than retrieving it from a server. Applets may have access to the browser's cache. Standalone applications can use the `java.net.ResponseCache` class. The `useCaches` variable determines whether a cache will be used if it's available. The default value is `true`, meaning that the cache will be used; `false` means the cache won't be used.

Timeouts

When your program connects to a website or server, it waits for a response. But what if the server is slow or not responding? You don't want your program to wait forever!

Timeouts help by setting a **maximum waiting time**. If there's no response within that time, your program stops waiting and gives an error (`SocketTimeoutException`).

There are **two types of timeouts**:

1. **Connection Timeout** → How long to wait while trying to connect to the server.
2. **Read Timeout** → How long to wait for data to arrive after connecting.

Methods to Set & Get Timeouts

- **setConnectTimeout (int timeout):** Sets the time to wait for connection.
- **getConnectTimeout():** Gets the current connection timeout value.
- **setReadTimeout(int timeout):** Sets the time to wait for data after connecting.
- **getReadTimeout():** Gets the current read timeout value.

Example:

```
import java.io.*;

import java.net.*;

public class TimeoutExample {

    public static void main(String[] args) {

        try {

            URL url = new URL("https://www.example.com/");

            URLConnection connection = url.openConnection();

            // Set timeouts (5 seconds for connection, 3 seconds for reading)

            connection.setConnectTimeout(5000);

            connection.setReadTimeout(3000);

            // Print timeout values

            System.out.println("Connect Timeout: " + connection.getConnectTimeout() + " ms");

            System.out.println("Read Timeout: " + connection.getReadTimeout() + " ms");

            // Open the connection and read data
```

```
        BufferedReader reader = new BufferedReader(new
InputStreamReader(connection.getInputStream()));

        System.out.println("Response: " + reader.readLine());

        reader.close();

    } catch (SocketTimeoutException e) {

        System.out.println("Request timed out!");

    } catch (IOException e) {

        e.printStackTrace();

    }

}

}
```

5.6 Configuring the Client Request HTTP Header

An HTTP client (e.g., a browser) sends the server a request line and a header. For example, here's an HTTP header that Chrome sends:

Accept:text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Accept-Charset:ISO-8859-1,utf-8;q=0.7,*;q=0.3

Accept-Encoding:gzip,deflate,sdch

Accept-Language:en-US,en;q=0.8

Cache-Control:max-age=0

Connection:keep-alive

Cookie:reddit_first=%7B%22firsttime%22%3A%20%22first%22%7D

DNT:1

Host:lesswrong.com

User-Agent:Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_3) AppleWebKit/537.31

(KHTML, like Gecko) Chrome/26.0.1410.65 Safari/537.31

A web server can use this information to serve different pages to different clients, to get and set cookies, to authenticate users through passwords, and more. Placing different fields in the header that the client sends and the server responds with does all of this.

5.7 Security Considerations for URLConnections

When a program tries to connect to a website or open a file using **URLConnection**, it has to follow certain security rules. These rules help protect your computer from unsafe or unauthorized actions.

For example: A web app (applet) can only connect to the website it was downloaded from. A program cannot randomly access your computer's file using a file URL.

To check whether a connection is allowed or blocked, Java provides a method:

public Permission getPermission() throws IOException

This method returns a Permission Object that tell us what kind of access is needed for the connection to work.

For example, if a program tries to connect www.example.com, Java will return a `SocketPermission` saying “This program needs permission to connect to www.example.com”. If no special permission is needed (like for public websites), the method returns null (meaning it’s safe to proceed).

5.8 Guessing MIME Media Types

When downloading files, program need to know what type of data they are receiving. This is usually indicated by MIME types (like `text/html`) for webpages or (`image/png` for images).

However, some servers don not provide correct MIME types or don’t provide them at all. TO handle this, Java provides two methods in `URLConnection` to guess the MIME type:

1. Guessing from File Name

- **Method:**
`public static String guessContentTypeFromName(String name)`
- **How it works?**
 - It looks at the file extension (e.g., `.jpg`, `.pdf`).
 - It uses a predefined list (found in `content-types.properties` in Java).
- **Example:**

```
String mimeType =  
URLConnection.guessContentTypeFromName("example.png");  
System.out.println(mimeType); // Output: image/png
```

2. Guessing from File Content (Stream)

- **Method:**
`public static String guessContentTypeFromStream(InputStream in)`
- **How it works?**
 - Reads the first 16 bytes of a file.
 - Matches against known patterns (e.g., PNG files start with `89 50 4E 47`).
- **Example:**

```
InputStream input = new FileInputStream("example.pdf");  
String mimeType = URLConnection.guessContentTypeFromStream(input);  
System.out.println(mimeType); // Output: application/pdf
```

5.9 HttpURLConnection

Java provides a special class called `HttpURLConnection` for working with HTTP requests. This class inherits from `URLConnection` but adds extra HTTP-specific features, such as:

- Setting request methods (e.g., GET, POST)
- Handling redirects (following URL changes)
- Getting response codes (e.g. 200 OK, 404 Not Found)
- Detecting proxy servers

We cannot create an `HttpURLConnection` directly because it is abstract (meaning we cannot use `new HttpURLConnection()`). Instead, Java automatically creates it when we open an <http://URL>. Here's how:

- **Step 1: Open a connection**

```
URL url = new URL("http://example.com");
URLConnection uc = url.openConnection();
```

But this only returns a generic `URLConnection`, which doesn't have HTTP-specific features.

- **Step 2: Convert it to HttpURLConnection: To get all the HTTP-specific methods, we case it like this:**

```
HttpURLConnection http = (HttpURLConnection) uc;
```

Or, we can do it in one step:

```
HttpURLConnection http = (HttpURLConnection) new
URL("http://example.com").openConnection();
```

The Request Method

When a web browser or client talks to a web server, the first thing it sends is a request line. This tells the server what action to perform, like retrieving a webpage, submitting a data, or deleting a file.

For example, if a browser wants to fetch a webpage, it sends this request:

```
GET /catalog/jfcnut/index.html HTTP/1.0
```

This means:

- GET: Request method (asking for a file)g/jfcnut/index.html: File being requested.

- HTTP/1.0: HTTP version being used.

By default, Java's `URLConnection` uses `GET`, but we can change it using:

```
http.setRequestMethod("METHOD_NAME");
```

This method must be one of these (case-sensitive):

METHOD	What it Does
GET	Retrieves data
POST	Sends data to the server
HEAD	Requests only the header, not the full content
PUT	Updates a file on the server
DELETE	Deletes a file from the server
OPTIONS	Asks what methods the server supports
TRACE	Debugging tool, returns the request itself

Disconnecting from the server

When using HTTP 1.1, the connection between a client and the server can be persistent, meaning multiple requests and responses can be sent over the same connection. This reduces the overhead of creating new connections for each request.

However, even though HTTP supports persistent connections, servers will eventually close the connection after a certain period of inactivity (usually around 5 seconds). Despite this, client should still close the connection when it's finished using it to free up resources.

By default `URLConnection` supports HTTP Keep-Alive, which means the connection is kept open for reuse when you make more requests to the same server. But once you're done with the server, you can explicitly disconnect from it using the `disconnect()` method.

```
public abstract void disconnect();
```

This method allows the client to break the connection once it's done with the server. If there are any open streams, the `disconnect()` method will close them as well.

Handling Server Responses

When a client sends an HTTP request to a server, the server responds with a response message that contains a status code and other details about the response. The first line of the response includes a numeric code and a message that indicate the outcome of the request.

Common Response Codes:

1. **200 OK** – The request document was successfully found and returned by the server.
Example:
HTTP/1.1 200 OK
Cache-Control: max-age=3, must-revalidate
Content-Type: text/html; charset=UTF-8
Server: Apache
Transfer-Encoding: chunked
2. **404 Not Found** – The URL requested no longer points to a document, meaning it doesn't exist on the server.
Example:
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=ISO-8859-1
Server: Apache
<h1>404 FILE NOT FOUND</h1>
3. **301 Moved Permanently** – The resource has permanently moved to a new location. The browser should automatically redirect to the new location.
Example:
HTTP/1.1 301 Moved Permanently
Location: <http://www.cafeaulait.org/>
Server: Apache

Getting Response Code and Message:

We can use the `URLConnection` class to accept the response code and message from the server:

- **`getResponseCode()`**: Returns the numeric response code as an integer.

Example:

```
int responseCode = httpURLConnection.getResponseCode();
```

- **getResponseMessage():** Returns the text message that follows the numeric code (like “OK” or “ Not Found”).

Example:

```
String responseMessage = httpURLConnection.getResponseMessage();
```

Using Constants for Response Codes:

We can check the response code against the constants defined in HttpURLConnection for better readability.

Example:

```
if (responseCode == HttpURLConnection.OK) {  
    System.out.println("Request was successful.");  
} else if (responseCode == HttpURLConnection.NOT_FOUND) {  
    System.out.println("Resource not found.");  
}
```

Here’s an example of how we can use HttpURLConnection to send a request and handle the server’s response:

```
URL url = new URL("http://www.example.com");  
HttpURLConnection connection = (HttpURLConnection) url.openConnection();  
connection.setRequestMethod("GET");
```

```
int responseCode = connection.getResponseCode();  
String responseMessage = connection.getResponseMessage();
```

```
System.out.println("Response Code: " + responseCode);  
System.out.println("Response Message: " + responseMessage);
```

Proxies

Many Users behind firewalls or using AOL, or other high-volume ISP's access the web through proxy servers. The `usingProxy()` method tells us whether the particular `URLConnection` is going through a proxy server:

```
public abstract boolean usingProxy()
```

It returns true if a proxy is being used, false if not. In some contexts, the use of proxy server may have security implications.

Streaming Mode

When sending data to a server using `URLConnection`, Java typically stores the entire request body in memory before sending it. This allows it to calculate the Content-length header, which tells the server how much data to expect. However, this approach is inefficient for large data uploads.

To solve this, Java provides two streaming modes that allow sending data efficiently without storing everything in memory first.

1. **Chunked Streaming mode:** Here, data is sent in small chunks instead of waiting for entire request to be ready. Each chunk has its own length, and Java automatically manages the Content-length. It is used if the total data size is unknown. To turn on chunked transfer encoding, just pass the size of the chunks you want to `setChunkedStreamingMode()` method before you connect the URL:

```
Public void setChunkedStreamingMode(int chunkLength);
```

2. **Fixed-Length Streaming mode:** Here, you have to specify the exact number of bytes to send. Java immediately starts streaming data instead of caching it. It is used when the exact data size is known in advance. If you know exactly how big your data is, pass that number to the `setFixedLengthStreamingMode()` method:

```
public void setFixedLengthStreamingMode(int contentLength)
```